The 3rd Universal Cup. Stage 19: Shenyang（2024沈阳区域赛JBDE）

J. Make Them Believe

```cpp
#include<bits/stdc++.h>
using namespace std;
#define endl "\n"
#define ll long long

map<int , string>mp_name;
int a[9];

void solve(){
    string s;
    int score;
    for (int i = 1; i <= 8; i ++)
    {
        cin >> s >> score;
        mp_name[score] = s;
        a[i] = score;
    }

    int sc_mx1 = max({a[1], a[2], a[3], a[4]});
    int sc_mx2 = max({a[5], a[6], a[7], a[8]});
    if(sc_mx1 > sc_mx2){
        cout << mp_name[sc_mx1] << " beats " << mp_name[sc_mx2] << endl;
    }else{
        cout << mp_name[sc_mx2] << " beats " << mp_name[sc_mx1] << endl;
    }
}

int main(){
        ios_base::sync_with_stdio(false);
        cin.tie(nullptr);
        cout.tie(nullptr);
        solve();
        return 0;
}
```

cpp运行

B. Magical Palette

分析：比较难搞的构造，但是多试一试会发现行和列都是等差数列可以解决问题，且需要满足gcd（n , m） == 1， 还需要单独处理一下特殊情况才能A。

```cpp
#include <bits/stdc++.h>
#define ll long long
using namespace std;

ll gcd(ll a, ll b)
{
    if (a == 0)
        return b;
    return gcd(b % a, a);
}

void solve()
{
    int n, m;
    cin >> n >> m;
    if (n == 1 && m == 1)
    {
        cout << "Yes" << endl;
        cout << 0 << endl;
        cout << 0 << endl;
        return;
    }
```

```cpp
    if (n == 1)
    {
        cout << "Yes" << endl;
        cout << 1 << endl;
        for (int i = 0; i < m; i++)
        {
            cout << i << " ";
        }
        cout << endl;
        return;
    }
    if (m == 1)
    {
        cout << "Yes" << endl;
        for (int i = 0; i < n; i++)
        {
            cout << i << " ";
        }
        cout << 1 << endl;
        cout << endl;
        return;
    }
    if (gcd(n, m) == 1)
    {
        cout << "Yes" << endl;
        for (int i = 0; i < n; i++)
        {
            cout << i * m + 1 << " ";
        }
        cout << endl;
        for (int j = 0; j < m; j++)
        {
            cout << j * n + 1 << " ";
        }
        cout << endl;
    }
    else
    {
        cout << "No" << endl;
    }
}

int main()
{

    int t;
    cin >> t;
    while (t--)
        solve();

    return 0;
}
```

cpp运行

D. Dot Product Game

稳铜的题，非常的不错值得练，难度不大。

题目大意：

- **玩家与操作**: Alice 和 Bob 玩游戏。Alice 只能操作排列 `A`，Bob 只能操作排列 `B`。Alice 先手。
- **有效操作**: 交换自己排列中的任意两个元素 `p_i` 和 `p_j`。
- **胜利条件**: 这个操作必须能**严格增大**两个排列的点积 `Σ a_i * b_i`。如果一个玩家无法做出任何能增大点积的交换，他就输了。
- **游戏性质**:
    - 这是一个**公平游戏 (Impartial Game)**，因为可行的操作只依赖于当前的 `A` 和 `B` 的状态，而与轮到谁操作无关。

- 玩家都足够聪明，会采用最优策略。
- 游戏保证会结束，因为点积有一个明确的上限（根据排序不等式，当 A 和 B 的顺序相同时，点积最大）。

游戏的总有效步数，就等于将 A 和 B 都调整到“有序”（即 A 和 B 相同）状态所需要的总步数。这个总步数的奇偶性，就等于 A 的逆序对数的奇偶性 加上 B 的逆序对数的奇偶性。

- **总逆序对数为奇数 -> 总步数为奇数 -> Alice赢**
- **总逆序对数为偶数 -> 总步数为偶数 -> Bob赢**
- 一次循环左移，需要 r-1 次相邻交换。
- d 次循环左移，就需要 d * (r-1) 次相邻交换。
- 每一次相邻交换都会让 ans 的奇偶性翻转一次。进行 d * (r-1) 次翻转，等价于给 ans 加上 d * (r-1)。

```
#include<bits/stdc++.h>
using namespace std;
#define endl "\n"
#define ll long long
// 一个排列通过交换操作达到有序状态所需的最少交换次数，与其逆序对数量的奇偶性相同。
const ll N = 500010;

ll tr[N];

ll n;

void init(ll n){
        for (ll i = 0; i <= n; i ++)
                tr[i] = 0;
}

void add(ll i , ll d){
        while(i <= n ){
                tr[i] += d;
                i += i & -i;
        }
}

ll sum(ll i){
        ll res = 0;
        while(i > 0 ){
                res += tr[i];
                i -= i & -i;
        }
        return res;
}

ll sum_range(ll l , ll r){
        return sum(r) - sum(l - 1);
}
void solve(){
        cin >> n ;
        vector<ll> a(n + 1), b(n + 1);
        ll ans = 0;
        for (ll i = 0; i < n ; i ++ ){
                cin >> a[i];
        }
        for (ll i = 0; i < n; i ++ ){
                cin >> b[i];
        }
        for (ll i = n - 1; i >= 0; i--){
                ans += sum_range(1, a[i] - 1);
                add(a[i], 1);
        }
        init(n);
        for (ll i = n - 1; i >= 0; i--)
        {
                ans += sum_range(1, b[i] - 1);
                add(b[i], 1);
        }

        // 计算出全部的 ans 的值 然后呢
        if(ans & 1)
```

```
                cout << "A";
        else
                cout << "B";
        for (ll i = 0; i < n - 1; i ++){
                char c;
                ll li, ri, di;
                cin >> c >> li >> ri >> di;
                ans += (ri - li) * di;
                if(ans & 1)
                cout << "A";
                else
                cout << "B";
        }
        cout << endl;
        init(n);
}

int main(){
        ios_base::sync_with_stdio(false);
        cin.tie(nullptr);
        cout.tie(nullptr);
        ll t;
        cin >>t;
        while(t--) solve();
        return 0;
}
```

cpp运行

E. Light Up the Grid

题目大意

题目要求在一个 2×2 的网格中设计一套固定的操作序列，使得无论实际初始网格的状态如何（题目给定若干个可能的初始状态），只要按照这个序列操作，总会在某个步骤使网格全亮（即所有单元为 1，从而触发提示音）。
网格中每个格子有两种状态（0 或 1），因此 2×2 网格共有 2^4=16 种可能的状态。题目提供了四种操作，每种操作会对网格中的若干格子进行“翻转”（0 变 1，1 变 0），且每种操作有对应的花费：

单格翻转：代价 a_0
整行翻转：代价 a_1
整列翻转：代价 a_2
全翻转：代价 a_3

这里采用状态压缩dp的方法解决。

```
#include <bits/stdc++.h>
using namespace std;
// #define endl "\n"
#define ll long long

const ll N = (1 << 16) + 10, M = 16; // 代表全部的状态
ll t, a0, a1, a2, a3;

ll dis[M][M]; // 记录最小的距离
ll dp[N][M], ans[N];
// 定义 dp[ms][j] 表示：经过一系列操作，已经覆盖了 ms 中所有初始状态，且最终操作结束后网格状态为 j 的最小总花费。
// 初始时，对每个 j ：dp[1 << j][j] = dis[j][15]

void add(ll x, ll y, ll w)
{
    dis[x][y] = min(dis[x][y], w);
    dis[y][x] = min(dis[y][x], w);
}

void init()
{

    memset(dis, 0x3f, sizeof dis);
    memset(dp, 0x3f, sizeof dp);

    for (ll i = 0; i < 16; i++)
```

```
    {
        add(i, i ^ 1, a0);
        add(i, i ^ 2, a0);
        add(i, i ^ 4, a0);
        add(i, i ^ 8, a0);
        add(i, i ^ 12, a1);
        add(i, i ^ 3, a1);
        add(i, i ^ 10, a2);
        add(i, i ^ 5, a2);
        add(i, i ^ 15, a3);
        dis[i][i] = 0; // 易错
    }

    dis[15][15] = 2 * min({a0, a1, a2, a3});

    for (ll k = 0; k < 16; k++)
    {
        for (ll i = 0; i < 16; i++)
        {
            for (ll j = 0; j < 16; j++)
            {
                dis[i][j] = min(dis[i][j], dis[i][k] + dis[k][j]); // 求出最短的距离开始dp
            }
        }
    }
    for (ll i = 0; i < 16; i++)
    {
        dp[1 << i][i] = dis[i][15];
    }
    for (ll mask = 0; mask < (1 << 16); mask++)
    {
        for (ll i = 0; i < 16; i++)
        {
            if (!((mask >> i) & 1))
                continue;
            for (ll j = 0; j < 16; j++)
            {
                if ((mask >> j) & 1)
                    continue;
                dp[mask | (1ll << j)][j] = min(dp[mask | (1ll << j)][j], dp[mask][i] + dis[j ^ (i ^ 15)][15]);
            }
        }
    }
    // dp[i][j] 就代表满足i的 最后到达15之前的状态是 j 的最小的花费
    memset(ans, 0x3f, sizeof ans);
    for (ll i = 0; i < (1 << 16); i++)
    {
        for (ll j = 0; j < 16; j++)
        {
            if ((i >> j) & 1)
                ans[i] = min(ans[i], dp[i][j]);
        }
    }
}

void solve()
{
    ll n;
    cin >> n;
    ll res = 0;
    ll cnt;
    for (ll i = 0; i < n; i++)
    {
        cnt = 0;
        string s1, s2;
        cin >> s1 >> s2;
        if (s1[0] == '1')
            cnt += 8;
        if (s1[1] == '1')
            cnt += 4;
        if (s2[0] == '1')
            cnt += 2;
        if (s2[1] == '1')
            cnt += 1;
        res |= (1 << cnt);
    }
    cout << ans[res] << endl;
}

int main()
{
```

```
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);
    cin >> t >> a0 >> a1 >> a2 >> a3;
    init();
    while (t--)
        solve();
    return 0;
}
```

cpp运行